Advanced Terraform on AWS Lab 3 Dynamic Resource Creation and Deterministic Design - Guided

Expected Duration 1
Code Review and Baseline Deployment 1
Replace Separate Subnet Variables with Structured Input 3
Refactor Subnets to use for_each 5
Refactor Route Table Associations to for_each 8
Test Deterministic Behaviour 10
Final Refactor – Introduce a Sanitization Layer 13
Lab Clean-up 15

Expected Duration

This lab should take 60-70 minutes to complete

Try to avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve may be lost and the lab learning value will therefore be reduced.

Code Review and Baseline Deployment

What You Are Trying to Do

Your objective is to understand the new networking components that have been introduced into the configuration, confirm that they deploy successfully, and then remove them in a controlled way.

Most of the supplied configuration follows the engineered pattern established in Lab 2. However, a new networking section has been added that does not follow these same standards.

1. Navigate to **labs\lab3**
2. Open **main.tf** and locate the following resources:
   - aws_subnet.app
   - aws_subnet.data
   - aws_route_table.main
   - aws_route_table_association.app

- aws_route_table_association.data

3. Compare these resources with the security group configuration carried over from Lab 2.
4. Consider:

   How are these resources defined?
   Are they driven by structured input?
   Is there any repetition?
   Do they follow the same pattern as the security group rules?

5. Open **variables.tf** and **terraform.tfvars** and locate the following variables:

   app_subnet_cidr
   data_subnet_cidr

6. Consider:

   What would happen if you needed to add a third subnet?
   How many changes would be required?
   Does this approach scale cleanly?

Deploy the Baseline and Verify Behaviour

7. Run the following commands:

   ```
   terraform init
   terraform plan
   terraform apply --auto-approve
   terraform state list
   ```

   You should see the creation of the same fifteen resources as were deployed in Lab2 along with five new resources:

   aws_subnet.app
   aws_subnet.data
   aws_route_table.main
   aws_route_table_association.app
   aws_route_table_association.data

   You do not need to perform deep validation. This step is to confirm that the configuration works as expected.

Controlled Rollback

8. Destroy only the five new networking resources introduced in this lab:

   ```
   terraform destroy `
     --target=aws_subnet.app `
     --target=aws_subnet.data `
     --target=aws_route_table_association.app `
     --target=aws_route_table_association.data `
     --target=aws_route_table.main `
   ```

```
--auto-approve
```

This will remove only the new networking resources, leaving the rest of the infrastructure intact.

Reflection

At this point, the configuration works correctly. However, the networking section does not follow the design patterns established in the previous lab because it relies on separate variables, introduces repetition, is not driven by structured input and does not scale cleanly.

Next you will begin to correct this by restructuring the input model.

Replace Separate Subnet Variables with Structured Input

What You Are Trying to Do

Whilst deploying correctly, the current implementation relies on separate variables for each subnet. This approach works for a small number of subnets, but it does not scale cleanly.

Every new subnet would require:

- A new variable
- Additional Terraform code
- Additional route table association code

In this stage, the goal is to improve the structure of the input data before changing the subnet resources themselves by moving from standalone subnet variables to a single structured subnet map.

You are not yet refactoring the subnet resources into a dynamic for_each design, at this stage, the focus is purely on improving the shape of the input model.

Replace the Existing Subnet Variables

9. Open variables.tf.
10. Locate and remove the existing subnet variables:

    variable "app_subnet_cidr"
    variable "data_subnet_cidr"

11. Add a new variable:

```
variable "subnet_cidrs" {
type = map(string)
description = "Map of subnet names to CIDR blocks."

validation {
condition = alltrue([
for cidr in values(var.subnet_cidrs) :
can(cidrhost(trimspace(cidr), 0))
```

```
])
error_message = "Each subnet CIDR must be a valid CIDR block."
}
}
```

This new variable allows multiple subnet CIDRs to be stored in a single structured object rather than separate standalone variables.

Replace the tfvars Entries

12. Open terraform.tfvars.
13. Locate the existing subnet CIDR values:

    app_subnet_cidr = "10.50.1.0/24"
    data_subnet_cidr = "10.50.2.0/24"

14. Replace them with:

```
subnet_cidrs = {
  app  = "10.50.1.0/24"
  data = "10.50.2.0/24"
}
```

The subnet definitions are now grouped into a single logical structure.

Update the Existing Subnet Resources

At this stage, the subnet resources themselves are still hard-coded. You are not refactoring them yet.

15. Open **main.tf**.
16. Locate the existing subnet resources:

    resource "aws_subnet" "app"
    resource "aws_subnet" "data"

17. Update the cidr_block values so they reference the new subnet map:

```
cidr_block = trimspace(var.subnet_cidrs["app"])
cidr_block = trimspace(var.subnet_cidrs["data"])
```

The resources remain separate resources for now. Only the input model has changed. The subnet resource addresses themselves also remain unchanged at this stage.

Verify the Behaviour

18. Run: **terraform plan**

The plan should propose the recreation of the five networking resources removed earlier. The change from separate subnet variables to a subnet map has not changed the intended infrastructure model. The same infrastructure is still being proposed, but the input structure is now more scalable.

Reflection

You have now improved the shape of the subnet input model without changing the behaviour of the deployed infrastructure. The configuration now stores subnet definitions in a structured map rather than individual standalone variables.

This is an important preparatory step toward a fully dynamic subnet design.

At this stage:

- The subnet resources are still repetitive
- The route table associations are still repetitive
- The configuration still does not scale cleanly

However, the input model is now moving toward the same structured design pattern used elsewhere in the codebase.

Refactor Subnets to use for_each

What You Are Trying to Do

In the previous stage, you replaced separate subnet variables with a single structured subnet map. However, the subnet resources themselves are still repetitive as the configuration still contains separate subnet resources. This means that adding a new subnet would still require additional Terraform code.

In this phase, you will refactor the subnet resources so that Terraform creates one subnet for each entry in the subnet map.

You are moving from hard-coded subnet resources to dynamic subnet resources using for_each

At this stage, the route table associations will remain separate resources. You will refactor those later.

Review the Existing Subnet Map Keys

19. Before refactoring the subnet resources, review the subnet map currently being used as input.
20. Run: terraform console
21. Inside the console, evaluate `keys(var.subnet_cidrs)`

    Expected output:

    ```
    [
      "app",
      "data",
    ]
    ```

    These keys already exist within the structured subnet map. However, the subnet resources themselves are still manually defined:

aws_subnet.app
aws_subnet.data

At this stage, the map keys are only acting as input data. They are not yet driving Terraform resource instances.

22. Exit the console: `exit`

Refactor the Subnet Resources

23. Open **main.tf**.
24. Locate the existing subnet resources:

    resource "aws_subnet" "app"
    resource "aws_subnet" "data"

25. Replace both subnet resources with the following single resource:

```
resource "aws_subnet" "subnet" {
  for_each = var.subnet_cidrs

  vpc_id            = aws_vpc.main.id
  cidr_block        = trimspace(each.value)
  availability_zone = "${var.region}a"

  tags = {
  Name = "${local.prefix}-subnet-${each.key}"
  }
}
```

    This creates one subnet for each entry in var.subnet_cidrs.

    The map keys become the stable resource instance keys:

    aws_subnet.subnet["app"]
    aws_subnet.subnet["data"]

Update the Route Table Associations

26. The route table association resources are still hard-coded, but they now need to point to the new subnet resource addresses.
27. Locate:

    resource "aws_route_table_association" "app"
    resource "aws_route_table_association" "data"

28. Update the subnet references so they use the new subnet instances:

```
subnet_id = aws_subnet.subnet["app"].id
subnet_id = aws_subnet.subnet["data"].id
```

Verify the Behaviour

29. Run: terraform plan

    The plan should still propose the creation of the same five networking resources.

    However, the subnet resource addresses have changed from:

    aws_subnet.app
    aws_subnet.data

    to:

    aws_subnet.subnet["app"]
    aws_subnet.subnet["data"]

    This is expected.

Review the Keys Again

30. Run: terraform console
31. Evaluate: keys(var.subnet_cidrs)

    Expected output:

    ```
    [
      "app",
      "data",
    ]
    ```

    The keys themselves have not changed. What has changed is how Terraform is now using them. Previously keys existed only as structured input. Now keys define Terraform resource instance identity. This is one of the most important concepts when using for_each.

32. Exit the console: `exit`

Reflection

You have now removed the repetitive subnet resource definitions and replaced them with a single for_each resource. The subnet model is now driven directly by the structured subnet map. At this stage, adding a new subnet no longer requires additional subnet resource definitions. However, the route table associations are still repetitive.

That is the next refactor target

Refactor Route Table Associations to for_each

What You Are Trying to Do

In the previous stage, you refactored the subnet resources into a dynamic for_each design. The subnet configuration is now scalable because Terraform creates subnet resources dynamically from the subnet map.

However, the route table associations are still manually defined:

resource "aws_route_table_association" "app"
resource "aws_route_table_association" "data"

This means that:

- adding a new subnet still requires additional association code
- the subnet configuration and association configuration are no longer aligned
- the configuration is only partially dynamic

In this phase, you will refactor the route table associations, so they are also created dynamically. You are moving from manual association resources to dynamic association resources. This is an important Terraform design pattern: Dependent resources should often scale automatically from the resources they depend on.

Refactor the Route Table Associations

33. Open **main.tf**.
34. Locate the existing route table association resources:

    resource "aws_route_table_association" "app"
    resource "aws_route_table_association" "data"

35. Remove both resources.
36. Replace them with:

```
resource "aws_route_table_association" "subnet" {
  for_each = aws_subnet.subnet

  subnet_id      = each.value.id
  route_table_id = aws_route_table.main.id
}
```

    This resource now creates one route table association for each subnet instance created by aws_subnet.subnet. Terraform is now chaining one dynamic resource collection directly into another.

Verify the Behaviour

37. Run: terraform plan

    The plan should still propose the same five networking resources.

However, the route table association resource addresses have now changed from:

aws_route_table_association.app
aws_route_table_association.data

to:

aws_route_table_association.subnet["app"]
aws_route_table_association.subnet["data"]

This is expected.

38. Apply the configuration: terraform apply --auto-approve

Review the Relationship

At this stage, the subnet keys now drive both:

aws_subnet.subnet["app"]
aws_route_table_association.subnet["app"]

and:

aws_subnet.subnet["data"]
aws_route_table_association.subnet["data"]

The subnet map keys are now controlling:

- subnet creation
- dependent resource creation
- Terraform resource instance identity

This is one of the most important scalability patterns when using for_each.

Reflection

You have now removed the repetitive route table association resources and replaced them with a single dynamic resource.

The subnet map now drives:

- Subnet resources
- Route table associations
- Dependent infrastructure relationships

At this stage:

- Adding a new subnet automatically creates a matching association
- The subnetting configuration is now dynamically scalable
- The networking model is now deterministic and pattern-driven

However, the configuration has not yet been tested against:

- Subnet reordering

- Subnet expansion
- Sloppy input formatting

That is the next stage.

Test Deterministic Behaviour

What You Are Trying to Do

In the previous stages, you refactored both the subnet resources and the route table associations into dynamic for_each resources. The networking configuration is now driven entirely from the structured subnet map.

At this stage, the configuration appears scalable. However, an important question still remains: Is the infrastructure behaviour stable and deterministic?

In this phase, you will test how Terraform behaves when:

- New subnets are added
- Subnet ordering changes
- Inconsistent formatting is introduced

This is one of the most important reasons for using for_each + map keys rather than count + positional indexes

The goal is to prove that Terraform resource identity is now tied to stable map keys rather than positional ordering.

Test 1 – Expand the Subnet Map

39. Open terraform.tfvars.
40. Add a third subnet:

```
subnet_cidrs = {
 app = "10.50.1.0/24"
 data = "10.50.2.0/24"
 mgmt = "10.50.3.0/24"
}
```

41. Run: terraform apply --auto-approve

    Terraform should create one additional subnet and one additional route table association. The existing subnet resources should remain unchanged. You should see new resource instances similar to:

    aws_subnet.subnet["mgmt"]
    aws_route_table_association.subnet["mgmt"]

    This demonstrates that adding a new subnet no longer requires:

    - additional subnet resource blocks

- additional association resource blocks
- additional Terraform duplication

Test 2 – Reorder the Map Entries

42. Reorder the subnet map:

```
subnet_cidrs = {
 mgmt = "10.50.3.0/24"
 app  = "10.50.1.0/24"
 data = "10.50.2.0/24"
}
```

The subnet definitions now appear in a completely different order.

43. Run terraform plan

Terraform should propose no changes. This is an extremely important result. Terraform is no longer using positional ordering to determine infrastructure identity. Instead, Terraform resource identity is now driven by stable map keys.

Test 3 – Introduce Sloppy Formatting

44. Modify the subnet map again, introducing inconsistent white-spacing:

```
subnet_cidrs = {
 mgmt = " 10.50.3.0/24 "
 data = "10.50.2.0/24"
 app  = " 10.50.1.0/24"
}
```

45. Run terraform plan

Terraform should still propose no changes. This demonstrates that ***trimspace(each.value)*** is preventing cosmetic formatting differences from leaking into the infrastructure model.

```
resource "aws_subnet" "subnet" {
  for_each          = var.subnet_cidrs
  vpc_id            = aws_vpc.main.id
  cidr_block        = trimspace(each.value)
  availability_zone = "${var.region}a"
  tags = {
    Name = "${local.prefix}-subnet-${each.key}"
  }
}
```

Reflection

You have now tested the networking configuration against:

- subnet expansion
- subnet reordering
- inconsistent formatting

The configuration now demonstrates deterministic behaviour because:

- subnet identity is driven by stable map keys

- dependent resources scale automatically
- positional ordering no longer affects infrastructure identity

This is one of the primary advantages of using for_each with maps rather than positional resource generation techniques such as count with lists

The networking configuration is now:

- scalable
- deterministic
- pattern-driven
- resistant to configuration churn

Final Refactor – Introduce a Sanitization Layer

What You Are Trying to Do

Throughout this course, you have been moving toward a more structured and maintainable Terraform design. The networking configuration now behaves correctly. It supports dynamic subnet creation, automatically creates dependent route table associations, and demonstrates deterministic behaviour when subnet ordering changes.

However, one small inconsistency remains.

The subnet CIDR values are currently sanitised directly within the subnet resource using: cidr_block = trimspace(each.value)

Whilst this works correctly, it mixes data-cleaning logic with resource-creation logic.

In previous labs, sanitisation was performed within a dedicated local value before being consumed by resources.

In this stage, you will move subnet sanitisation into a dedicated sanitization layer so that resources consume pre-cleaned data rather than cleaning data themselves.

This creates a more consistent design pattern across the codebase and makes the resource definitions easier to read and maintain.

Create a Sanitized Subnet Map

46. Open **locals.tf**

47. Add the following local value:

```
sanitized_subnet_cidrs = {
 for subnet_name, cidr in var.subnet_cidrs :
  trimspace(subnet_name) => trimspace(cidr)
 }
```

This local value creates a cleansed version of the original subnet map by removing any leading or trailing whitespace from both subnet names and CIDR values.

Update the Subnet Resource

48. Open **main.tf**
49. Locate: resource "aws_subnet" "subnet"
50. Replace:

    for_each = var.subnet_cidrs

    with:

    for_each = local.sanitized_subnet_cidrs

51. Replace:

    cidr_block = trimspace(each.value)

    with:

    cidr_block = each.value

    The resource no longer performs sanitisation itself. It now consumes already-sanitized data.

Verify the Behaviour

52. Run: **terraform plan**

    Terraform should propose no changes.

    The infrastructure behaviour remains identical because the sanitisation logic has simply been moved into a dedicated preprocessing layer.

Reflection

You have now completed the final refinement of the networking configuration.

The configuration now follows a consistent pattern:

Input Variables
↓
Validation
↓
Sanitization Layer
↓
Resource Creation

At this stage the networking configuration is:

- Scalable through dynamic resource creation
- Deterministic through stable map keys
- Resistant to formatting inconsistencies
- Consistent with the design patterns established elsewhere in the codebase

This separation of responsibilities is a common Terraform engineering practice because it keeps resource definitions focused on infrastructure creation while data preparation is handled independently.

Lab Clean-up

53. Destroy all resources created during this lab; **terraform destroy --auto-approve**

Congratulations, you have completed this lab

Copyright